

2026



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

EVALUACIÓN EN CONTACTO CON EL DOCENTE

MSC. LILIAN AMAN

DIEGO CÓRDOVA

Tabla de contenido

1. Planificación del proyecto 2

2. Diseño del sistema 4

3. Desarrollo del software 8

4. Integración de contenidos.....13

1. Planificación del proyecto

A. Nombre del proyecto (integra contenido de cuatro unidades):

El impacto de las nuevas tecnologías en la sociedad ha dejado de ser una promesa de futuro para convertirse en el motor que redefine nuestra realidad. En este escenario, el **desarrollo y proyección de soluciones informáticas** es el puente crítico entre las necesidades humanas y la capacidad técnica para resolverlas.

Ya no hablamos solo de digitalizar procesos, sino de transformar la estructura misma de la salud, la economía, la educación y la sostenibilidad.

B. Descripción del problema

Para abordar el impacto de las nuevas tecnologías mediante un proyecto informático, el primer paso es identificar problemas reales en la sociedad, como la gestión ineficiente del tiempo o la falta de automatización en tareas cotidianas. A partir de ahí, se definen casos de uso específicos (por ejemplo, un sistema que controle el acceso a un servicio o un asistente que organice tareas pendientes o), los cuales determinan cómo interactuará el usuario con la solución. Finalmente, estos casos de uso se traducen en código mediante la programación, donde se utilizan variables y listas para almacenar y organizar los datos, combinadas con estructuras lógicas como if-else para tomar decisiones en tiempo real, y bucles while o for para repetir procesos y recorrer la información de manera eficiente.

C. Desarrollar un sistema funcional basado en una problemática definida.

La Problemática: "Fatiga del aprendizaje pasivo"

En entornos educativos o corporativos, memorizar glosarios, términos técnicos o vocabulario de un nuevo idioma suele ser aburrido y poco efectivo cuando solo se lee un documento.

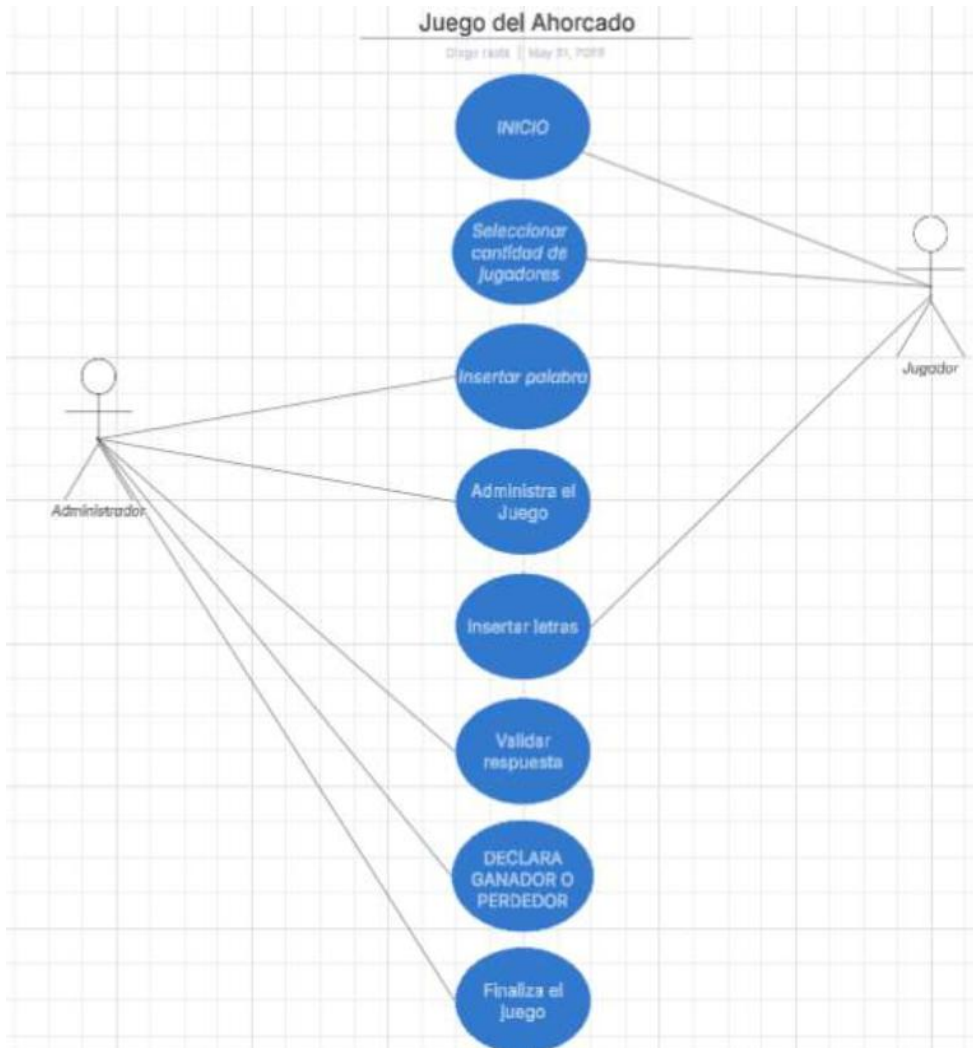
La Solución: "Ahorcado Educativo Interactivo"

Convertiremos el clásico juego del ahorcado en una herramienta de aprendizaje activo. El sistema permitirá al usuario adivinar palabras técnicas basadas en **pistas conceptuales**, reforzando la retención mediante el juego.

S1	S2	S3	S4	S5	S6	S7	S8
Unidad 1		Unidad 2		Unidad 3		Unidad 4	
Tema 1 Introducción a la Resolución de Problemas y al Entorno de Programación	Tema 2 Entorno de programación	Tema 3 Manejo de Datos	Tema 4 Algoritmos y Diagramas de flujo	Tema 5 Condicionales	Tema 6 Bucles	Tema 7 Estructura de Datos y Funciones	Tema 8 Entrega del proyecto final de Evaluación en Contacto con el Docente
El proyecto se desarrollará durante las 8 semanas del curso y deberá seguir las siguientes fases:							
Aprendizaje Autónomo 1 de temas 1 y 2		Aprendizaje Autónomo 2 de temas 3 y 4		Temas 3, 4 y 5		Repositorio en GitHub	
1. Análisis del Problema y Casos de Uso (Diseño Funcional) Antes de dibujar o programar, debes delimitar qué acciones puede realizar el usuario (Actor) y cómo reacciona el sistema. En el Ahorcado, los casos de uso principales que debes plasmar en tu diagrama funcional son: 2. Flujo Lógico del Sistema (Diagrama de Flujo) Como parte de tu diseño funcional, necesitas un diagrama que muestre la secuencia algorítmica del juego de principio a fin. Este flujo debe representar visualmente los bucles y las condiciones del software: 3. Arquitectura del Software (Componentes Principales) Para el diagrama de arquitectura, debes dividir el juego en bloques o capas independientes para demostrar orden de software, incluso si es un programa pequeño. Una arquitectura limpia para el Ahorcado.		1. Control de Flujo con Estructuras Repetitivas (El Bucle Principal) Para cumplir con la implementación de bucles, el juego del Ahorcado necesita un ciclo principal que mantenga la partida activa. 2. Toma de Decisiones mediante Estructuras Lógicas (Condicionales Anidados) Las condicionales (If / Else) representan el cerebro de la lógica del juego y determinan el destino del jugador en cada turno. 3. Organización, Modularidad y Documentación del Código Para asegurar que el código sea claro, estructurado y fácil de evaluar y de verificar, no debes escribir todo en un solo bloque continuo.		1. Uso explícito del Bucle While: Controla el ciclo de vida del juego basándose en una condición lógica dinámica (vidas > 0). El juego no sabe cuántas veces va a fallar el usuario, por lo que este bucle es el ideal. 2. Uso del Bucle For: Se emplea dentro de la función mostrar tablero. Es una estructura determinada porque sabemos exactamente cuántas letras tiene la palabra secreta y la recorreremos de principio a fin de forma secuencial. 3. Comentarios de Arquitectura: El código está documentado indicando qué parte pertenece a los Datos (lista de palabras), cuál a la Interfaz (imprimir el tablero) y cuál al Core o Lógica (bucle while, control de vidas y condiciones de fin).		• Código completo del proyecto • Diagramas desarrollados (funcionalidad y arquitectura) • Archivo README con: • Nombre del proyecto • Integrantes • Objetivo del sistema • Descripción de funcionalidades • Fecha	

2. Diseño del sistema

Elaborar diagramas de funcionalidad (casos de uso, flujo, etc.).



Los **casos de uso** se basan en la representación de los requerimientos funcionales de un sistema. En términos sencillos: describen **quién** (el Actor) interactúa con el sistema y **qué** acciones o tareas (los Casos de Uso) puede realizar en él para lograr un objetivo específico.

Basándonos en la imagen de tu diagrama de un "Juego del Ahorcado", aquí tienes el detalle estructurado en **4 puntos importantes**:

1. Actores Claramente Definidos

El diagrama identifica las dos entidades externas que interactúan con el software:

- **Administrador (Izquierda):** Es el actor que tiene un control técnico y de gestión sobre el flujo del juego (cargar datos, supervisar y cerrar el sistema).
- **Jugador (Derecha):** Es el usuario final, el participante que interactúa directamente con la mecánica interactiva del juego.

2. Flujo Secuencial del Juego (Ciclo de Vida)

Aunque los diagramas de casos de uso no muestran estrictamente el orden temporal, los óvalos de este diseño están estructurados visualmente para representar el ciclo de vida completo de una partida:

- Comienza desde un estado inicial (**INICIO**).
- Pasa por la fase de configuración (**Seleccionar cantidad de jugadores, Insertar palabra**).

- Llega a la ejecución central (**Insertar letras**).
- Concluye con la resolución del juego (**DECLARA GANADOR O PERDEDOR, Finaliza el juego**).

3. División de Responsabilidades (Roles)

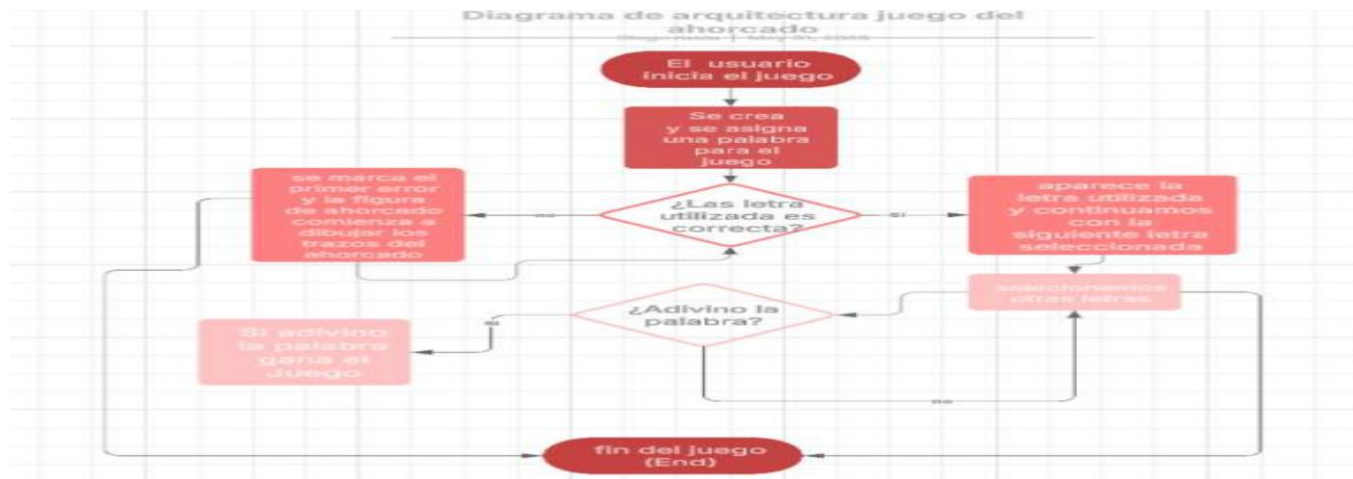
Existe una separación de tareas muy marcada en el sistema:

- **Tareas exclusivas del Administrador:** Administrar el juego, validar las respuestas del usuario de forma directa o indirecta, declarar el estado final (ganador/perdedor) y finalizar el proceso de manera segura.
- **Tareas del Jugador:** Iniciar la app, elegir con cuántas personas jugar e introducir activamente las letras para intentar adivinar.
- **Puntos de encuentro:** El caso de uso "**Insertar palabra**" está extrañamente conectado a ambos actores (lo que sugiere que un jugador puede proponer la palabra secreta para que otro la adivine, o bien que el administrador la digita).

4. Lógica de Control del Sistema

El diagrama refleja que el juego no se maneja de forma automática por completo; depende de un rol de control de fondo (**Administrador**).

Casos de uso como "**Validar respuesta**" y "**Administra el juego**" nos indican que el backend o la lógica de negocio está fuertemente ligada a acciones de supervisión para asegurar que las reglas del ahorcado (conteo de vidas, aciertos y fallos) se cumplan correctamente antes de emitir un veredicto.



Diseñar la arquitectura del sistema.

Estructura de Componentes de la Arquitectura

1. Admin (Administrador / Configuración del Sistema)

Es el punto de origen del sistema. Representa al programador o al archivo de configuración que establece los parámetros iniciales del juego antes de que se ejecute.

- **Función en el Ahorcado:** Definir el banco o repositorio de palabras secretas disponibles y establecer el límite predeterminado de intentos fallidos (5 vidas).

2. Código (Lógica de Negocio / Core del Juego)

Es el motor lógico del juego (</>). Recibe las instrucciones iniciales del administrador y procesa el estado interno del juego de forma matemática y abstracta.

- **Función en el Ahorcado:** Ejecutar el bucle principal (while), verificar si la letra ingresada por el usuario coincide con la palabra secreta, actualizar la lista de letras usadas y restar las vidas correspondientes.

3. Imagen (Controlador de Estado Visual / Render)

Es la capa intermedia encargada de transformar las variables lógicas y numéricas puras provenientes del **Código** en representaciones gráficas o de texto comprensibles.

- **Función en el Ahorcado:** Tomar el número de vidas actuales y la lista de letras correctas para armar dinámicamente el dibujo del monito y la cadena de guiones bajos (ej. A _ O _ _).

4. Jugador (UI - Interfaz de Usuario) y Usuario

Representa el entorno de interacción final. La **UI** es la ventana de la terminal o consola, mientras que el **Usuario** es el jugador humano que camina a través de la experiencia del juego.

- **Función en el Ahorcado:** La UI proyecta en la pantalla lo que preparó el componente "Imagen" y abre el canal de captura de datos (input) para que el usuario introduzca su letra por teclado.

5. Resultado (Módulo de Cierre / Desenlace)

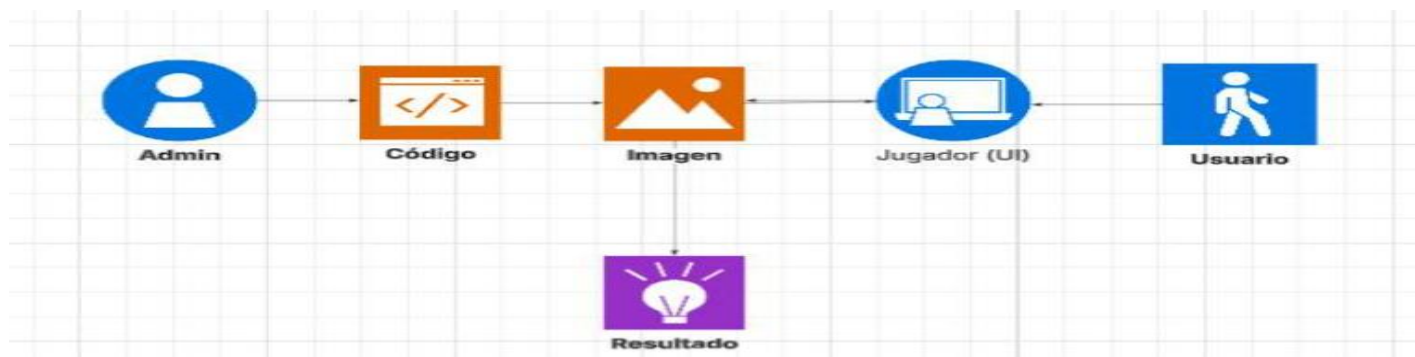
Es el componente que se dispara como consecuencia directa de la última actualización del estado visual de la partida.

- **Función en el Ahorcado:** Evaluar las condiciones de victoria o derrota inmediatas para detener el ciclo del juego y desplegar la pantalla final correspondiente (Mensaje de "¡Felicidades, ganaste!" o "Game Over").

Flujo de Datos según tus Flechas Directrices

Para tu documento o video, puedes explicar el flujo continuo de tu diagrama de la siguiente manera:

1. El **Admin** inicializa el diccionario de palabras que el **Código** va a procesar.
2. El **Código** evalúa matemáticamente el turno actual y le envía los datos crudos a la **Imagen**.
3. La **Imagen** procesa el dibujo correspondiente y se lo envía a la interfaz del **Jugador (UI)** para que el **Usuario** lo vea y reaccione ingresando una nueva letra.
4. Cuando la **Imagen** detecta que ya no quedan guiones por pintar o que el monito del ahorcado está completo, redirige el flujo hacia el **Resultado** final para cerrar la ejecución del programa.



3. Desarrollo del software

Implementar el sistema en base a los diagramas diseñados

```
ejercicio3.py ejerciciosSemana5.py autonomo_2.py ultimacase.py Evaluacion_Docente.py X
C:\Users\kathy\Desktop\Python> Evaluacion_Docente.py
1 # 1. Configuración inicial de jugadores
2 jugadores = [] # Lista que guardará las tuplas (numero, nombre)
3 contador_jugadores = 0
4
5 print("=== CONFIGURACIÓN DE JUGADORES ===")
6
7 while True:
8     print(f"\nJugadores actuales en la lista: {jugadores}")
9     print(f"Total de jugadores registrados: {contador_jugadores}")
10
11     opcion = input("\n¿Qué deseas hacer? (1: Agregar / 2: Quitar / 3: Iniciar Juego): ")
12
13     if opcion == "1":
14         nombre = input("Ingresa el nombre del jugador: ").capitalize()
15         contador_jugadores += 1
16         # Creamos una tupla con el número y el nombre
17         nuevo_jugador = (contador_jugadores, nombre)
18         jugadores.append(nuevo_jugador)
19         print(f"Jugador {nombre} agregado con el ID {contador_jugadores}!")
20
21     elif opcion == "2":
22         if len(jugadores) == 0:
23             print("No hay jugadores para quitar.")
24         else:
25             nombre_quitar = input("Ingresa el nombre del jugador a quitar: ").capitalize()
26             # Buscamos al jugador en la lista de tuplas
27             encontrado = False
28             for jugador in jugadores:
29                 if jugador[1] == nombre_quitar:
30                     jugadores.remove(jugador)
31                     print(f"Jugador {nombre_quitar} eliminado.")
32                     encontrado = True
33                     break
34             if not encontrado:
35                 print("No se encontró ningún jugador con ese nombre.")
36
37     elif opcion == "3":
38         if len(jugadores) == 0:
39             print("¡Debes agregar al menos un jugador para comenzar!")
40         else:
41             print("\n¡Lista de juego cerrada!")
42             break
43     else:
44         print("Opción no válida. Intenta de nuevo.")
45
46 # 2. Configuración del juego de palabras
47 print("\n" + "="*30)
48 palabra_secreta = input("Ingresa la palabra secreta para el juego: ").lower()
```

adelante.

1. Implementación Basada en el Diagrama de Casos de Uso

Un diagrama de casos de uso define las interacciones de los actores (los jugadores o el administrador) con el sistema. Tu código traduce esto secuencialmente en dos bloques de control principales (while):

Caso de Uso 1: Gestionar Jugadores (Administrador / Jugadores)

Representa la primera fase del código. Permite preparar el entorno antes de jugar:

- **Sub-caso: Agregar Jugador:** Implementado en la condición `opcion == "1"`. El sistema recibe la entrada, genera un ID único incremental y formatea el nombre.
- **Sub-caso: Quitar Jugador:** Implementado en `opcion == "2"`. Realiza una validación previa (que haya elementos) y ejecuta una búsqueda iterativa eliminando por coincidencia de texto.
- **Sub-caso: Validar e Iniciar:** Implementado en `opcion == "3"`. El sistema actúa como guardián impidiendo el avance si la lista jugadores está vacía.

Caso de Uso 2: Configurar Juego (Moderador)

- **Ingresar Palabra Secreta:** Línea 48. El sistema fuerza la entrada a minúsculas (`.lower()`) para evitar errores de disparidad de caracteres más

Caso de Uso 3: Jugar Partida (Jugadores)

Representa el bucle principal de juego. Contiene los siguientes flujos del diagrama:

- **Visualizar Progreso:** El sistema evalúa el estado de las letras descubiertas frente a `palabra_secreta`.
- **Arriesgar Letra:** Valida las precondiciones del negocio (que sea un único carácter y que sea alfabético: `.isalpha()`).
- **Verificar Letra Repetida:** Controla que no se penalice ni premie doble por una opción ya intentada.
- **Controlar Intentos (Fallo/Éxito):** Evalúa el fin del juego mediante dos caminos alternativos (Ganar si `palabra_completa == True` o Perder si `contador_errores == intentos_maximos`).

2. Implementación Basada en la Arquitectura del Sistema

Si comparamos este software con un modelo arquitectónico estándar, tu script se clasifica bajo una **Arquitectura Monolítica de una Sola Capa (Scripting / CLI)**.

Capa de Datos (Persistencia Volátil)

Toda la persistencia de la información es local y ocurre en la memoria RAM asignada al proceso de Python. Si el script se cierra, los datos se destruyen:

- `jugadores`: Estructura de tipo **Lista de Tuplas** `[(int, str)]`.
- `letras_adivinadas`: Estructura de tipo **Lista** `[str]`.

Capa de Presentación (Frontend / UI)

Es una interfaz basada en texto (CLI - *Command Line Interface*). Depende completamente de:

- Las funciones nativas de entrada/salida de la consola (`print` e `input`).
- El formateo de strings en línea utilizando interpolación (f-strings) para renderizar dinámicamente las variables a la vista del usuario.

Aplicar estructuras lógicas (condicionales y repetitivas)

```
tion View Go Run ... Search
ejercicio3.py ejerciciosSemana5.py autonomo_2.py ultimacase.py EvaluacionDo
C > Users > kathy > Desktop > Python > Evaluacion_Docente.py > ...
49 intentos_maximos = 5
50 contador_errores = 0
51 letras_adinadas = []
52
53 print(f"\nEl juego comienza! Participantes: {[j[1] for j in jugadores]}")
54 print(f"Tienes {intentos_maximos} oportunidades para fallar.")
55
56 # 3. Bucle principal
57 while contador_errores < intentos_maximos:
58     print("\n" + "="*30)
59
60     # Mostrar el progreso de la palabra
61     palabra_completa = True
62     for letra in palabra_secreta:
63         if letra in letras_adinadas:
64             print(letra, end=" ")
65         else:
66             print("_", end=" ")
67             palabra_completa = False
68
69     print("\n")
70
71     if palabra_completa:
72         print("¡GANADORES! Han adivinado la palabra secreta.")
73         break
74
75     # Solicitar una letra al jugador
76     letra_ingresada = input("Adivine una letra: ").lower()
77
78     if len(letra_ingresada) != 1 or not letra_ingresada.isalpha():
79         print("Por favor, ingresa solo una letra.")
80         continue
81
82     if letra_ingresada in letras_adinadas:
83         print("Ya habias ingresado esa letra, intenta con otra.")
84
85     elif letra_ingresada in palabra_secreta:
86         print(f"¡Bien hecho! La letra '{letra_ingresada}' es correcta.")
87         letras_adinadas.append(letra_ingresada)
88
89     else:
90         contador_errores += 1
91         print(f"Error número: {contador_errores}")
92         print(f"Te quedan {intentos_maximos - contador_errores} oportunidades.")
93         letras_adinadas.append(letra_ingresada)
94
95 # 4. Fin del juego
96 if contador_errores == intentos_maximos:
```

1. Estructuras Repetitivas (Bucles)

Sirven para ejecutar un bloque de código varias veces de manera automática.

A. Ciclo while True (Línea 7)

- **Dónde está:** Controla la "Configuración inicial de jugadores".
- **Para qué sirve:** Crea un bucle infinito para mostrar el menú (Agregar, Quitar, Iniciar) una y otra vez. Se rompe con un break (Línea 42) al elegir la opción 3.

B. Ciclo for jugador in jugadores: (Línea 28)

- **Dónde está:** Dentro de la opción de quitar un jugador (opcion == "2").
- **Para qué sirve:** Recorre la lista de jugadores uno por uno para comprobar si el nombre ingresado coincide con alguno de los guardados.

C. Ciclo while contador_errores < intentos_maximos: (Línea 55)

- **Dónde está:** Al inicio del bucle principal del juego.
- **Para qué sirve:** Mantiene el juego activo mientras el usuario tenga vidas disponibles. Si los errores igualan los intentos máximos, el bucle termina.

D. Ciclo for letra in palabra_secreta: (Línea 60)

- **Dónde está:** Al principio de la visualización de la palabra.
- **Para qué sirve:** Recorre cada letra de la palabra secreta para decidir si dibuja la letra

real (si ya fue adivinada) o un guion bajo _ (si sigue oculta).

2. Estructuras Condicionales

Sirven para tomar decisiones. El programa ejecuta un camino u otro según si la condición es verdadera (True) o falsa (False).

A. Estructura if / elif / else del Menú (Líneas 12 - 43)

- **Dónde está:** En la configuración de jugadores.
- **Para qué sirve:** Enruta las opciones del menú: "1" agrega, "2" elimina, "3" inicia el juego, y cualquier otra cosa (else) avisa que la opción es inválida.

B. Condicional de Validación de Lista Vacía (Líneas 21 y 37)

- **Dónde está:** Al intentar quitar un jugador o iniciar el juego.
- **Para qué sirve:** Actúa como regla de seguridad. Impide borrar a alguien si la lista está vacía (Línea 21) o empezar a jugar sin participantes registrados (Línea 37).

C. Condicional de Letra Correcta o Incorrecta (Líneas 82 - 93)

- **Dónde está:** En el corazón del bucle del juego.
- **Para qué sirve:** Procesa la jugada del usuario:
 - **if:** Filtra errores (más de una letra o caracteres no válidos).
 - **elif (86):** Evita penalizar si la letra ya se había adivinado.
 - **elif (89):** Si la letra es correcta, la agrega a los aciertos.
 - **else:** Si no es ninguna anterior, cuenta como fallo, suma un error y resta una vida.

D. Condicional de Fin de Juego por Derrota (Línea 99)

- **Dónde está:** Al final del código.
- **Para qué sirve:** Determina el desenlace. Si el juego terminó por falta de intentos, este if se activa para mostrar el mensaje de derrota y revelar la palabra secreta.

Organizar el código de manera clara y estructurada.

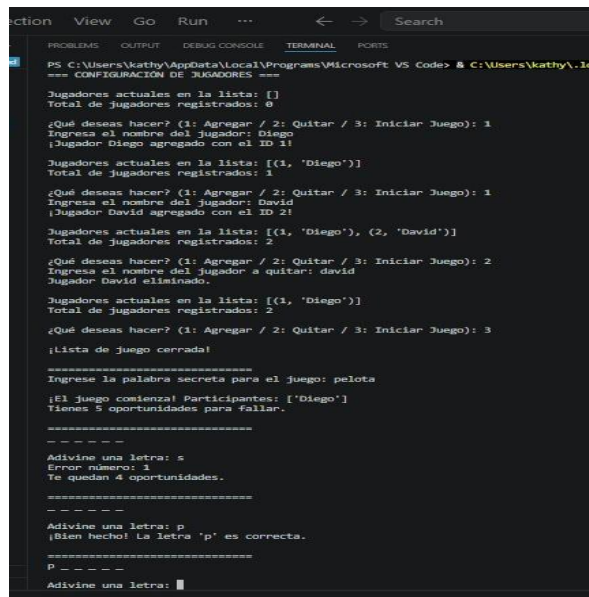
Como observamos en nuestro código lo tenemos organizado como se aprecia en el repositorio de GitHub teniendo en cuenta lo siguiente.

La idea principal y objetiva de organizar el código de manera clara y estructurada es **garantizar la mantenibilidad, escalabilidad y legibilidad del software a lo largo del tiempo.**

En términos de ingeniería, esto se traduce en pasar de un código lineal (tipo "sábana") a una arquitectura donde **cada componente tiene una sola responsabilidad clara.**

4. Integración de contenidos

Funcionalidades del sistema



```
PS C:\Users\kathy\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kathy\...  
==== CONFIGURACIÓN DE JUGADORES ====  
Jugadores actuales en la lista: []  
Total de jugadores registrados: 0  
¿Qué deseas hacer? (1: Agregar / 2: Quitar / 3: Iniciar Juego): 1  
Ingresar el nombre del jugador: Diego  
¡Jugador Diego agregado con el ID 1!  
Jugadores actuales en la lista: [(1, 'Diego')]  
Total de jugadores registrados: 1  
¿Qué deseas hacer? (1: Agregar / 2: Quitar / 3: Iniciar Juego): 1  
Ingresar el nombre del jugador: David  
¡Jugador David agregado con el ID 2!  
Jugadores actuales en la lista: [(1, 'Diego'), (2, 'David')]  
Total de jugadores registrados: 2  
¿Qué deseas hacer? (1: Agregar / 2: Quitar / 3: Iniciar Juego): 2  
Ingresar el nombre del jugador a quitar: david  
Jugador David eliminado.  
Jugadores actuales en la lista: [(1, 'Diego')]  
Total de jugadores registrados: 1  
¿Qué deseas hacer? (1: Agregar / 2: Quitar / 3: Iniciar Juego): 3  
¡Lista de juego cerrada!  
Ingresar la palabra secreta para el juego: pelota  
¡El juego comienza! Participantes: ['Diego']  
Tienes 5 oportunidades para fallar.  
-----  
Adivine una letra: s  
Error número: 1  
Te quedan 4 oportunidades.  
-----  
Adivine una letra: p  
¡Bien hecho! La letra 'p' es correcta.  
-----  
P -----  
Adivine una letra: █
```

La funcionalidad del sistema representa el "qué hace" el programa desde la perspectiva del usuario final. Reúne todo lo que hemos analizado: los casos de uso (los objetivos), la arquitectura (el soporte técnico), las estructuras lógicas (las reglas) y las herramientas.

1. Módulo de Gestión y Registro de Participantes

El sistema actúa inicialmente como un panel de administración en memoria. Su función es preparar la lista de las personas que van a interactuar en la sesión de juego.

Registro dinámico: Permite introducir nombres de jugadores uno a uno. El sistema limpia la entrada (convierte la primera letra en mayúscula) y le asigna un identificador numérico único secuencial (ID).

Modificación de lista: Permite eliminar usuarios en caso de error o retiro antes de empezar. El sistema busca de manera inteligente en la lista y remueve al participante exacto.

Control de acceso: Bloquea el inicio del juego si no se cumple el requisito mínimo del negocio (contar con al menos un jugador registrado).

2. Inicialización del Entorno de Juego

Una vez cerrada la lista de participantes, el sistema cambia de estado y prepara las variables secretas que gobernarán la partida.

Establecimiento del objetivo: Permite ingresar la palabra que se tendrá que adivinar, transformándola por completo a minúsculas para asegurar un juego justo e inmune a errores de mayúsculas.

Apertura de marcadores: Inicializa el contador de errores en cero, define el límite estricto de fallos (5 oportunidades) y crea una lista vacía para registrar el historial de letras que se vayan utilizando.

3. Motor del Ciclo de Juego (Gameplay Loop)

Esta es la funcionalidad central y más compleja del sistema. Es un ciclo interactivo que procesa las acciones del usuario de forma repetitiva:

Renderizado de Progreso: En cada turno, el sistema escanea la palabra secreta. Muestra las letras que ya fueron descubiertas y oculta las demás bajo el símbolo _.

Validación de Entradas: Actúa como un filtro estricto. Si el usuario ingresa números, símbolos o más de una letra a la vez, el sistema rechaza la jugada sin penalizarlo y exige una nueva entrada.

Evaluación de Impacto (Lógica de Juego): * Si la letra es correcta, se guarda en el historial de aciertos y se revela en la palabra sin restar oportunidades.

Si la letra es incorrecta, el sistema aumenta el contador de errores y notifica visualmente cuántas vidas le quedan al grupo.

Si la letra ya fue usada, el sistema advierte al jugador para que no pierda oportunidades en vano.

4. Resolución y Determinación del Desenlace

El sistema evalúa constantemente el tablero de juego en cada vuelta del ciclo para determinar de forma automática cuándo y cómo termina la sesión de software:

Condición de Victoria: Si tras ingresar una letra el sistema detecta que ya no quedan caracteres ocultos (_), detiene el bucle inmediatamente y despliega el mensaje de felicitación declarando a los participantes como GANADORES.

Condición de Derrota: Si el contador de errores alcanza el límite de 5 vidas, el sistema corta la ejecución, bloquea nuevos intentos, anuncia el Fin del Juego y revela de forma transparente la palabra secreta en la pantalla.

Estructura lógica del programa

1. Estructura Secuencial (El Camino Lineal)

Las instrucciones se ejecutan de arriba hacia abajo, una tras otra, en orden estricto.

En tu código: El salto entre bloques principales. Pasa obligatoriamente por el Bloque 1 (Configuración de jugadores), luego al Bloque 2 (Configuración de palabras) y finalmente al juego. No puede saltarse pasos ni ir directo a adivinar.

2. Estructura Selectiva o Condicional (La Toma de Decisiones)

El programa bifurca su camino. Evalúa si una condición es Verdadera o Falsa y ejecuta un bloque de código u otro según el resultado.

En tu código:

Selección Múltiple (if/elif/else): Menú de jugadores. Si digitas "1" va a registrar, si digitas "2" va a eliminar. Los caminos son excluyentes.

Validaciones de Seguridad: El condicional `if len(jugadores) == 0:`. Si es verdadero, frena el avance y muestra un error; si es falso, permite continuar.

3. Estructura Repetitiva o Iterativa (Los Bucles)

Permite repetir un bloque de instrucciones de forma automática para no duplicar código. Se divide en dos tipos:

Ciclos controlados por condición (while): Se ejecutan mientras una condición sea verdadera.

En tu código: El bucle principal del juego (`while contador_errores < intentos_maximos:`). El programa repite turnos de forma elástica hasta que el jugador se queda sin vidas o gana.

Ciclos controlados por colección (for): Tienen un fin definido. Recorren una estructura elemento por elemento.

En tu código: El renderizado de la palabra (`for letra in palabra_secreta:`). Si la palabra tiene 5 letras, da exactamente 5 vueltas para decidir si dibuja la letra o un guion _.

Uso de herramientas de desarrollo

El **uso de herramientas de desarrollo** (comúnmente llamadas *DevTools*) consiste en aprovechar un conjunto de aplicaciones, utilidades y plataformas de software especializadas que permiten a los programadores crear, probar, depurar (eliminar errores) y optimizar sus aplicaciones de manera eficiente.

En el desarrollo moderno, no se programa "a ciegas". Las herramientas actúan como el panel de control de un avión: te muestran exactamente qué está pasando dentro del sistema, en la memoria RAM, en el procesador o en el tráfico de red.

Reflexionar sobre el uso y futuro de la tecnología en el entorno real.

1. El Uso Actual: De la Automatización a la Omnipresencia

En el entorno real actual, la tecnología cumple una función principal: **eliminar la fricción**.

- **Resolución de problemas cotidianos:** El código que hoy analizamos como un juego de consola (el ahorcado) comparte la misma lógica base (bucles, condicionales, estructuras de datos) que los sistemas que gestionan turnos en un hospital, controlan el inventario de un supermercado o validan las credenciales de acceso en una aplicación bancaria.

2. El Futuro Inmediato: Tendencias en el Entorno Real

El futuro de la tecnología no se trata solo de hacer computadoras más rápidas, sino de hacerlas más humanas y adaptables.

A. Inteligencia Artificial Ubicua y Cooperativa

Ya no solo interactuamos con software estático que sigue reglas fijas (if/else). El futuro pertenece a sistemas capaces de aprender del contexto en tiempo real, predecir fallos mecánicos antes de que ocurran en las fábricas o personalizar la educación de un estudiante según su ritmo de aprendizaje.

B. Arquitecturas Distribuidas y Computación en la Nube

Los programas ya no corren aislados en una sola máquina (como el script en tu computadora). El entorno real exige sistemas conectados globalmente a través de microservicios y APIs, donde los datos viajan de forma segura entre servidores de todo el mundo en milisegundos.

Conclusión: El Rol del Desarrollador del Futuro

La tecnología en el entorno real es tan buena como la lógica y la empatía de quien la diseña.

(<https://uide.instructure.com/courses/34196/modules>, s.f.)

(<https://blog.corporacionbi.com/productos-servicios/el-impacto-del-cambio-tecnologico-en-la-sociedad>, s.f.) (Córdova, s.f.)

(Córdova, https://github.com/Diego250896/Evaluaci-n-en-Contacto-con-el-Docente/blob/main/Evaluacion_Docente.py, s.f.)

Bibliografía

Córdova, D. (s.f.). *file:///C:/Users/Katherine/Desktop/Diego%20UIDE/Aprendizaje%20Autonomo%20N1.pdf*.

Córdova, D. (s.f.). *https://github.com/Diego250896/Evaluaci-n-en-Contacto-con-el-Docente/blob/main/Evaluacion_Docente.py*.

https://about.netflix.com/es_es. (s.f.).

<https://blog.corporacionbi.com/productos-servicios/el-impacto-del-cambio-tecnologico-en-la-sociedad>. (s.f.).